

# FOUNDATION\_DROOLS - Business Rules Engine Foundation

Developer implementation guide: FOUNDATION\_DROOLS\_DEV\_IMPL\_GUIDE.md.

## 1. Purpose

This foundation defines how SOPHIX uses Drools/KIE Server to execute configurable business rules outside normal application code.

Use this document when implementing:

- Rule-driven validation.
- Eligibility decisions.
- Pricing or fee applicability rules.
- Dunning escalation rules.
- Workflow precondition checks.
- Operator-specific rule packages.
- KJAR build/deploy/versioning.

## 2. Ownership

FOUNDATION\_DROOLS owns:

- KIE Server deployment topology.
- KJAR packaging conventions.
- Rule invocation contract.
- Fact/result modeling conventions.
- Rule versioning and deployment rules.
- Error handling and monitoring expectations.

FOUNDATION\_DROOLS does not own:

- The business meaning of rules. Each DD owns its own rule logic.
- Rule facts for a specific module. The owning module defines them.
- Workflow orchestration. See FOUNDATION\_CAMUNDA.md.
- API authentication. See FOUNDATION\_AUTH.md.

## 3. When To Use Drools

Use Drools when rules are:

- Business-owned or likely to change by operator/country.
- A decision table or many conditional rules.
- Needed consistently by multiple code paths.
- Better audited by rule ID than hidden in imperative code.
- Operator-specific, for example KE WIK vs WUG vs YASSN.

Do not use Drools for:

- Simple field validation.
- Database constraints.
- One-off `if` statements unlikely to vary.

- High-frequency inner loops where a normal code branch is cheaper and clearer.

## 4. Runtime Architecture

```

flowchart LR
    JAVA[Java modules] -->|HTTPS JSON| LB[NGINX or LB]
    PHP[Laravel modules] -->|HTTPS JSON| LB
    LB --> K1[KIE Server node 1]
    LB --> K2[KIE Server node 2]
    K1 --load KJAR--> REPO[(Maven / artifact repo)]
    K2 --load KJAR--> REPO

```

KIE Server is stateless for SOPHIX calls. Each request sends facts in and receives rule results out. The caller owns persistence and business action.

## 5. Deployment Topology

| Item           | Requirement                                                                               |
|----------------|-------------------------------------------------------------------------------------------|
| Runtime        | Drools 8 / KIE Server                                                                     |
| Nodes          | 2 KIE Server nodes                                                                        |
| Front door     | NGINX or load balancer                                                                    |
| State          | Stateless request/response                                                                |
| Rule artifacts | KJARs loaded from Maven/artifact repository                                               |
| Auth           | Basic auth or internal service auth over TLS                                              |
| Endpoint       | <code>https://kie.sophix.internal/kie-server/services/rest/serve</code><br><code>r</code> |

If one KIE node fails, the load balancer routes to the surviving node. There is no per-request session state to recover.

## 6. KJAR Layout

Each module that owns rules owns a rules artifact.

Recommended layout:

```

modules/{module}-rules/
  pom.xml
  src/main/resources/
    META-INF/kmodule.xml
    rules/
      {dd-code}-{purpose}.drl
      {dd-code}-{operator}-{purpose}.drl
  src/main/java/com/sophix/{module}/facts/
    RequestFact.java
    ContextFact.java
    DecisionResult.java
    ValidationError.java

```

`pom.xml` packaging:

```
<packaging>kjar</packaging>
```

`kmodule.xml`:

```

<kmodule xmlns="http://www.drools.org/xsd/kmodule">
  <kbase name="subscription-rules" packages="com.sophix.subscription.rules">
    <ksession name="subscription-session" default="true"/>
  </kbase>
</kmodule>

```

Rule file example:

```

package com.sophix.subscription.rules

import com.sophix.subscription.facts.ActivationRequestFact
import com.sophix.subscription.facts.ValidationError

rule "R-SUB-ACT-001 subscription must be CREATED"
when
    $req: ActivationRequestFact(statusCode != "CREATED")
then
    insert(new ValidationError(
        "R-SUB-ACT-001",

```

```

        "statusCode",
        "Subscription must be CREATED before activation"
    ));
end

```

## 7. Laravel-Owned Rules

Laravel modules may own KJARs without writing Java application code. Define fact types in DRL with `declare`.

```

package com.sophix.fulfillment.rules

declare PaymentFact
    amount    : java.math.BigDecimal
    expected  : java.math.BigDecimal
end

declare PaymentDecision
    classification : String
    amountShort    : java.math.BigDecimal
end

rule "R-FUL-PAY-001 classify partial payment"
when
    $p: PaymentFact(amount < expected)
then
    insert(new PaymentDecision("PARTIAL", $p.getExpected().subtract($p.getAmount())));
end

```

This keeps the HTTP contract the same for PHP and Java callers.

## 8. Container Naming and Versioning

Container ID pattern:

```
{module}-rules-{operator}_{version}
```

Examples:

```

subscription-rules-wik_1.0.0
billing-rules-wik_1.2.0
workorder-rules-wik_1.0.0

```

Versioning rules:

| Rule         | Description                                                                                  |
|--------------|----------------------------------------------------------------------------------------------|
| DROOLS-VER-1 | Version KJARs semantically: MAJOR.MINOR.PATCH.                                               |
| DROOLS-VER-2 | Patch version for rule bug fix that preserves facts/results.                                 |
| DROOLS-VER-3 | Minor version for additive facts/results or new rules.                                       |
| DROOLS-VER-4 | Major version for breaking fact/result changes.                                              |
| DROOLS-VER-5 | Caller config must pin the container ID. Do not silently float production callers to latest. |
| DROOLS-VER-6 | Keep older KJAR versions available while workflows that started on them may still run.       |

## 9. HTTP Invocation Contract

Endpoint:

```

POST /kie-server/services/rest/server/containers/instances/{containerId}
Authorization: Basic <internal>
Content-Type: application/json
X-KIE-ContentType: JSON

```

Request:

```

{
  "lookup": "subscription-session",
  "commands": [
    {
      "insert": {
        "object": {
          "com.sophix.subscription.facts.ActivationRequestFact": {
            "subscriptionId": "sub_01HX",
            "statusCode": "CREATED",

```

```

        "operatorCode": "WIK"
    },
    "out-identifier": "request"
  },
  { "fire-all-rules": { "max": -1 } },
  { "get-objects": { "out-identifier": "results" } }
]
}

```

Response:

```

{
  "type": "SUCCESS",
  "result": {
    "execution-results": {
      "results": {
        "results": {
          "value": []
        }
      }
    }
  }
}

```

Caller rules:

| Rule          | Description                                                                            |
|---------------|----------------------------------------------------------------------------------------|
| DROOLS-CALL-1 | Callers must set a request timeout. Recommended default: 3 seconds.                    |
| DROOLS-CALL-2 | Callers must log containerId, lookup, correlationId, and rule result summary.          |
| DROOLS-CALL-3 | Do not log sensitive full fact payloads unless redacted.                               |
| DROOLS-CALL-4 | Treat KIE unavailable as a dependency failure unless the owning DD defines a fallback. |
| DROOLS-CALL-5 | Map validation failures to BUSINESS_RULE_ERROR with rule ID.                           |

## 10. Standard Result Types

Use explicit result facts.

Validation error:

```

{
  "com.sophix.common.rules.ValidationError": {
    "ruleId": "R-SUB-ACT-001",
    "field": "statusCode",
    "message": "Subscription must be CREATED before activation"
  }
}

```

Decision:

```

{
  "com.sophix.common.rules.DecisionResult": {
    "decisionCode": "PAY_FIRST_REQUIRED",
    "ruleId": "R-BIL-001",
    "attributes": {
      "amount": "1500.00",
      "currency": "KES"
    }
  }
}

```

Rules:

| Rule         | Description                                                               |
|--------------|---------------------------------------------------------------------------|
| DROOLS-RES-1 | Every failure/decision result must include ruleId.                        |
| DROOLS-RES-2 | Result classes must be stable within a KJAR major version.                |
| DROOLS-RES-3 | Multiple validation errors may be returned in one call.                   |
| DROOLS-RES-4 | Caller decides whether errors are blocking or advisory based on DD rules. |

## 11. Java Caller Example

```
Map<String, Object> body = Map.of(
    "lookup", "subscription-session",
    "commands", List.of(
        Map.of("insert", Map.of(
            "object", Map.of(
                "com.sophix.subscription.facts.ActivationRequestFact",
                Map.of(
                    "subscriptionId", "sub_01HX",
                    "statusCode", "CREATED",
                    "operatorCode", "WIK"
                )
            ),
            "out-identifier", "request"
        )),
        Map.of("fire-all-rules", Map.of("max", -1)),
        Map.of("get-objects", Map.of("out-identifier", "results"))
    )
);

HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create(kieBaseUrl + "/containers/instances/subscription-rules-wik_1.0.0"))
    .header("Content-Type", "application/json")
    .header("X-KIE-ContentType", "JSON")
    .header("Authorization", basicAuth)
    .timeout(Duration.ofSeconds(3))
    .POST(HttpRequest.BodyPublishers.ofString(json.writeValueAsString(body)))
    .build();
```

## 12. Laravel Caller Example

```
$response = Http::withBasicAuth(config('rules.user'), config('rules.password'))
    ->timeout(3)
    ->withHeaders([
        'Content-Type' => 'application/json',
        'X-KIE-ContentType' => 'JSON',
    ])
    ->post(config('rules.base_url') . '/containers/instances/fulfillment-rules-wik_1.0.0', [
        'lookup' => 'fulfillment-session',
        'commands' => [
            [
                'insert' => [
                    'object' => [
                        'com.sophix.fulfillment.rules.PaymentFact' => [
                            'amount' => '50000.00',
                            'expected' => '60000.00',
                        ],
                    ],
                ],
                'out-identifier' => 'payment',
            ],
            ['fire-all-rules' => ['max' => -1]],
            ['get-objects' => ['out-identifier' => 'results']],
        ],
    ]);

$results = $response->json('result.execution-results.results.results.value', []);
```

## 13. Error Handling

| Condition                     | Caller behavior                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------------|
| KIE HTTP timeout              | Return dependency failure or retry according to owning DD.                              |
| KIE returns non-2xx           | Treat as dependency failure.                                                            |
| KIE response type not SUCCESS | Treat as rule-engine failure.                                                           |
| ValidationError result        | Return BUSINESS_RULE_ERROR or route workflow to rejection path.                         |
| Empty results                 | Usually means no rule rejected and no positive decision needed. Confirm with owning DD. |
| Unknown result class          | Treat as integration error; alert.                                                      |

Recommended API error:

```
{
  "error": {
```

```

    "code": "BUSINESS_RULE_ERROR",
    "message": "Activation request failed rule validation.",
    "ruleId": "R-SUB-ACT-001",
    "field": "statusCode",
    "correlationId": "corr_01HX"
  }
}

```

## 14. Testing

Required tests:

| Test type         | Requirement                                             |
|-------------------|---------------------------------------------------------|
| Rule unit tests   | Each DRL package has test facts for pass/fail paths.    |
| Contract tests    | Caller verifies expected result classes and JSON paths. |
| Integration tests | Staging caller hits staging KIE Server container.       |
| Regression tests  | New KJAR version runs old fixture set before deploy.    |
| Performance smoke | High-volume rules checked for acceptable latency.       |

## 15. Deployment Flow

1. Developer updates DRL/facts.
2. CI runs rule unit tests.
3. CI builds KJAR.
4. CI publishes KJAR to artifact repository.
5. KIE Server loads or reloads the container version.
6. Staging callers run integration tests.
7. Production caller config is updated to the new container ID.
8. Monitor rule latency and error rate after rollout.

Production rule: do not overwrite an existing KJAR version. Publish a new version.

## 16. Monitoring

Alert on:

| Signal                          | Threshold                       |
|---------------------------------|---------------------------------|
| KIE node count                  | Less than 2                     |
| Rule invocation p95 latency     | Above module-specific threshold |
| KIE HTTP 5xx                    | Any sustained increase          |
| KIE timeout rate                | Any sustained increase          |
| Container load failures         | Any                             |
| Rule result integration errors  | Any                             |
| Artifact repository unavailable | Any during deploy window        |

## 17. Medium Budget Infrastructure

### 17.1 Server Dimensions

Required medium baseline:

| Server role         | Count                   | CPU           | RAM       | Disk               | Notes                                  |
|---------------------|-------------------------|---------------|-----------|--------------------|----------------------------------------|
| KIE Server node     | 2                       | 2-4 vCPU each | 8 GB each | 80-100 GB SSD each | Stateless app nodes.                   |
| Artifact repository | Shared platform service | 2 vCPU        | 4-8 GB    | 100 GB+            | Maven/Nexus/Artifactory or equivalent. |
| Load balancer       | 1 minimum, 2 preferred  | 1-2 vCPU      | 2 GB      | 20 GB SSD          | NGINX/LB if on-prem.                   |

| Server role            | Count                   | CPU    | RAM  | Disk                 | Notes                        |
|------------------------|-------------------------|--------|------|----------------------|------------------------------|
| Monitoring/log storage | Shared platform service | 2 vCPU | 4 GB | Depends on retention | Stores KIE logs and metrics. |

Minimum acceptable start:

```
2 x KIE nodes: 2 vCPU / 8 GB RAM / 80 GB SSD
1 x LB node: 1-2 vCPU / 2 GB RAM / 20 GB SSD
```

Preferred medium:

```
2 x KIE nodes: 4 vCPU / 8-16 GB RAM / 100 GB SSD
2 x LB nodes: 1-2 vCPU / 2 GB RAM / 20 GB SSD
```

Placement rules:

- Do not place both KIE nodes on the same physical host.
- Do not co-locate KIE nodes with PostgreSQL primaries if avoidable.
- Keep KIE endpoint private to SOPHIX modules.
- Artifact repository should be backed up; rules are deployable artifacts.

## 17.2 Recommended OS

Use one operator-standard Linux distribution:

| OS                                   | Recommendation                                           |
|--------------------------------------|----------------------------------------------------------|
| Ubuntu Server 22.04 LTS or 24.04 LTS | Recommended default for medium teams.                    |
| Rocky Linux 9 / RHEL 9               | Use if the operator standardizes on RHEL-family systems. |

OS baseline:

- 64-bit minimal server install.
- No desktop packages.
- chrony or equivalent NTP enabled.
- Security patching through operator maintenance process.
- SSH restricted to admin VPN/network.
- Host firewall enabled.
- Java runtime pinned and tested with selected Drools/KIE version.

## 17.3 Disk Partitions

For each KIE node with 80-100 GB disk:

```
/          30 GB
/opt/kie   20 GB
/var/log   20 GB
/tmp       10 GB
free/reserved remaining
```

Partition rules:

- KIE nodes should not store durable business state.
- Logs must not fill /.
- KJARs are cached locally but source of truth is the artifact repository.

## 17.4 JVM Starting Point

For 8 GB RAM nodes:

```
JAVA_OPTS="-Xms2g -Xmx4g"
```

For 16 GB RAM nodes:

```
JAVA_OPTS="-Xms4g -Xmx8g"
```

Tune after observing rule latency, GC, and heap usage.

## 17.5 Network Ports

| Flow                            | Port                   | Exposure                          |
|---------------------------------|------------------------|-----------------------------------|
| Module to KIE LB                | 443                    | Internal application network only |
| LB to KIE node                  | 8080 or 8443           | Internal only                     |
| KIE node to artifact repository | 443 or repository port | Internal only                     |
| Admin SSH                       | 22                     | Admin VPN/network only            |
| Monitoring scrape               | Operator standard      | Monitoring network only           |

## 18. AWS Deployment Recommendations

This section applies only when deploying KIE Server on AWS.

### 18.1 Recommended AWS Shape

| Component           | AWS service                                                | Recommendation                                                                                       |
|---------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| KIE runtime         | ECS on Fargate, ECS on EC2, or EC2 Auto Scaling Group      | ECS/Fargate is preferred if the team already uses containers. EC2 ASG is simpler for VM-based teams. |
| Load balancer       | Internal Application Load Balancer                         | Keep KIE private to SOPHIX services.                                                                 |
| Artifact storage    | CodeArtifact, S3-backed repo, Nexus/Artifactory on EC2/ECS | Must support Maven artifacts.                                                                        |
| Shared file storage | EFS only if repository/runtime requires shared files       | KIE itself should load artifacts from repository, not depend on shared disk.                         |
| Secrets             | AWS Secrets Manager or SSM Parameter Store                 | Store KIE credentials and repository credentials.                                                    |
| Logs                | CloudWatch Logs                                            | Ship KIE logs.                                                                                       |

### 18.2 Compute Sizing

ECS/Fargate medium start:

Desired tasks: 2  
Task size: 2 vCPU / 8 GB memory  
Placement: Spread across 2 Availability Zones

EC2 ASG medium start:

2 x EC2 instances: m7g.large or m6i.large  
EBS: 80-100 GB gp3 each

Preferred medium:

2 x tasks/nodes: 4 vCPU / 8-16 GB memory

Use Graviton only after verifying the chosen KIE/Drools image and JDK work cleanly on ARM. Otherwise use x86 instances.

### 18.3 AWS Network Layout

Recommended VPC layout:

Private app subnets:  
- KIE tasks/instances  
- SOPHIX module services  
  
Private artifact subnet/service:  
- Artifact repository if self-hosted  
  
Public subnets:  
- No KIE resources

Security group rules:

| Source                | Destination                       | Port             | Purpose             |
|-----------------------|-----------------------------------|------------------|---------------------|
| Module service SG     | Internal ALB SG                   | 443              | Rule invocation     |
| ALB SG                | KIE task/EC2 SG                   | 8080 or 8443     | Forward traffic     |
| KIE SG                | Artifact repo SG/service endpoint | 443 or repo port | Pull KJARs          |
| Admin VPN/bastion/SSM | KIE nodes                         | 22 or SSM        | Admin access if EC2 |



## 18.4 AWS Operational Notes

- Use an internal ALB, not public.
- Use ECS task health checks or ALB health checks.
- Store rule artifacts in a Maven-compatible repository.
- Roll out KJAR versions through CI/CD, not by manual copy.
- Use CloudWatch alarms for unhealthy tasks, 5xx rate, latency, and task restarts.
- If using Fargate, choose valid CPU/memory combinations and size at the task level.

Reference docs:

- [AWS Fargate task CPU and memory]([https://docs.aws.amazon.com/en\\_us/AmazonECS/latest/developerguide/fargate-tasks-services.html](https://docs.aws.amazon.com/en_us/AmazonECS/latest/developerguide/fargate-tasks-services.html))
- [Use an Application Load Balancer for Amazon ECS](<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/alb.html>)
- [Amazon EFS documentation](<https://aws.amazon.com/documentation-overview/efs/>)

## 19. Developer Checklist

- [ ] Rule is justified as configurable/business-owned logic.
- [ ] KJAR has stable container ID and version.
- [ ] Rule names include rule IDs.
- [ ] Facts/results are versioned and contract-tested.
- [ ] Caller pins container version.
- [ ] Caller timeout is configured.
- [ ] Sensitive facts are not logged raw.
- [ ] Validation errors map to BUSINESS\_RULE\_ERROR.
- [ ] Integration tests call staging KIE Server.
- [ ] New KJAR version is published; old version is not overwritten.